

Class 10: Structural Bioinformatics Pt. 1

Aarav Prasad (PID: A17440940)

Background

The main repository of high-resolution structural data on biomolecules is called the **Protein Data Bank** (PDB).

PDB statistics

What is in the PDB in terms of molecule type and structure determination method?

Read a CSV file of current PDB stats obtained from <https://www.rcsb.org/stats/summary>

```
pdb <- read.csv("Data Export Summary.csv")
pdb
```

	Molecular.Type	X.ray	EM	NMR	Integrative	Multiple.methods
1	Protein (only)	180,758	23,111	12,813	348	229
2	Protein/Oligosaccharide	10,488	3,741	34	8	11
3	Protein/NA	9,205	6,751	287	26	8
4	Nucleic acid (only)	3,154	250	1,578	3	15
5	Other	178	27	35	4	0
6	Oligosaccharide (only)	11	0	6	0	1
	Neutron Other Total					
1	84 32	217,375				
2	1 0	14,283				
3	0 0	16,277				
4	3 1	5,004				
5	0 0	244				
6	0 4	22				

Q1: What percentage of structures in the PDB are solved by X-Ray and Electron Microscopy.

```
pdb$X.ray
```

```
[1] "180,758" "10,488" "9,205" "3,154" "178" "11"
```

This print out above `pdb$X.ray` is “character” not “numeric”. Therefore I can’t do math with it.

```
# We want to get rid / sub out commas:
```

```
x<- pdb$X.ray
tmp <- sub(",", "", x)
sum(as.numeric(tmp))
```

```
[1] 203794
```

We could make a function to do this:

```
rm.comma <- function(x) {
  tmp <- sub(",", "", x)
  sum(as.numeric(tmp))
}
```

```
rm.comma(pdb$EM)
```

```
[1] 33880
```

We could also use a different import function for this CSV that speaks American (i.e. deals with commas in numbers in a comma separated value file).

```
library(readr)

pdb <- read_csv("Data Export Summary.csv")
```

```
Rows: 6 Columns: 9
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
chr (1): Molecular Type
```

```
dbl (4): Integrative, Multiple methods, Neutron, Other
```

```
num (4): X-ray, EM, NMR, Total
```

```
i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
pdb
```

```
# A tibble: 6 x 9
```

	`Molecular Type` <chr>	`X-ray` <dbl>	EM <dbl>	NMR <dbl>	Integrative <dbl>	`Multiple methods` <dbl>	Neutron <dbl>
1	Protein (only)	180758	23111	12813	348	229	84
2	Protein/Oligosacch~	10488	3741	34	8	11	1
3	Protein/NA	9205	6751	287	26	8	0
4	Nucleic acid (only)	3154	250	1578	3	15	3
5	Other	178	27	35	4	0	0
6	Oligosaccharide (o~	11	0	6	0	1	0

```
# i 2 more variables: Other <dbl>, Total <dbl>
```

```
n.tot <- sum(pdb$Total)
```

```
n.xray <- sum(pdb$`X-ray`)
```

```
n.em <- sum(pdb$`EM`)
```

```
xray_pct <- n.xray / n.tot * 100
```

```
em_pct <- n.em / n.tot * 100
```

```
xray_pct
```

```
[1] 80.48577
```

```
em_pct
```

```
[1] 13.38046
```

Q2: What proportion of structures in the PDB are protein? How many total proteins?

```
pdb$Total[1]
```

```
[1] 217375
```

```
pdb$Total[1] / n.tot
```

```
[1] 0.8584941
```

The total number of protein sequences in Uniprot is 202,556,314

```
217375 / 202556314 * 100
```

```
[1] 0.1073158
```

Q3: Type HIV in the PDB website search box on the home page and determine how many HIV-1 protease structures are in the current PDB?

There are ~1200 HIV-1 protease structures in the current PDB

Key-point: We have a very, very small structural coverage of known proteins (0.1%). Most structures we know about (80%) come from one method (X-ray crystallography)

Visualizing PDB data with Mol-star

Main stand alone web version with all features is at <https://molstar.org/viewer/>



Figure 1: The HIV-protease enzyme is a homodimer with two chains

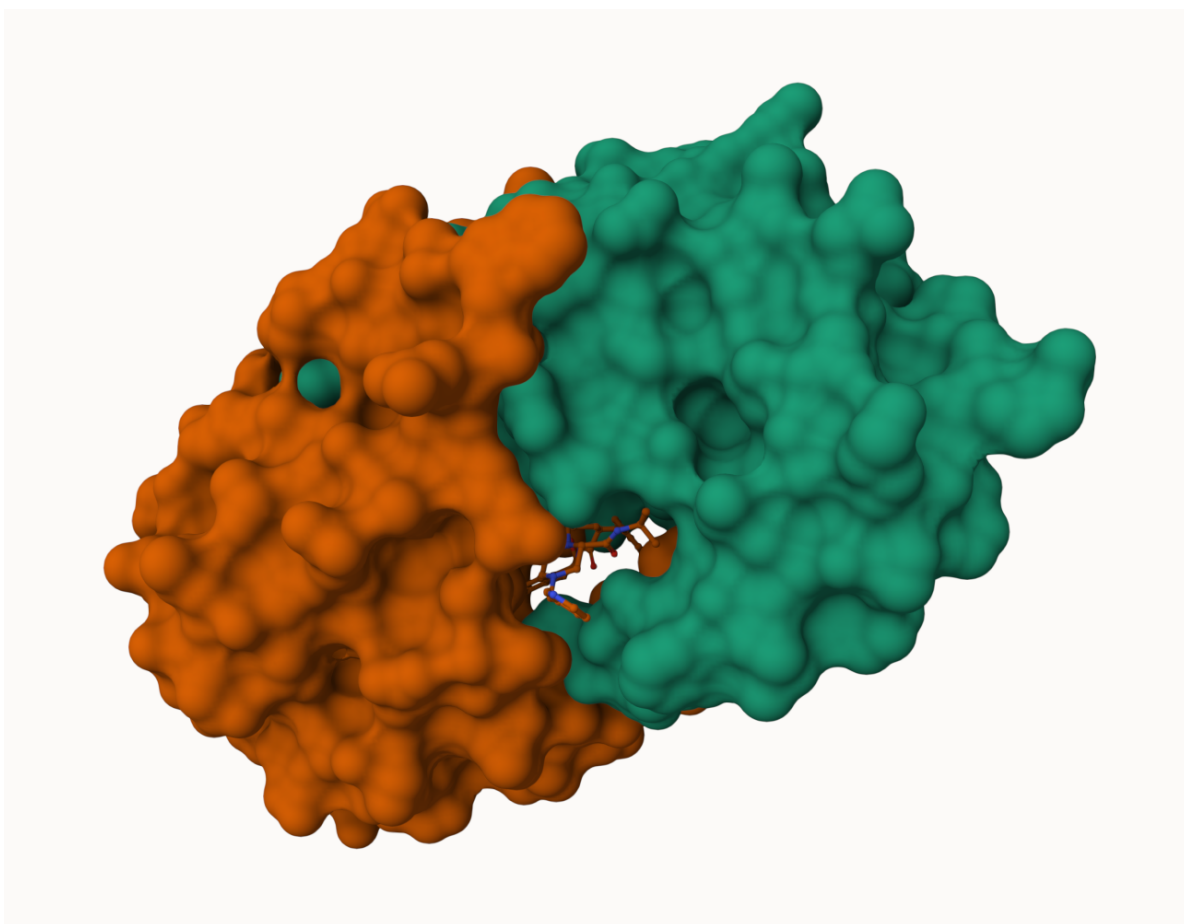


Figure 2: Surface display showing the binding cleft site for the inhibitor (drug) molecule

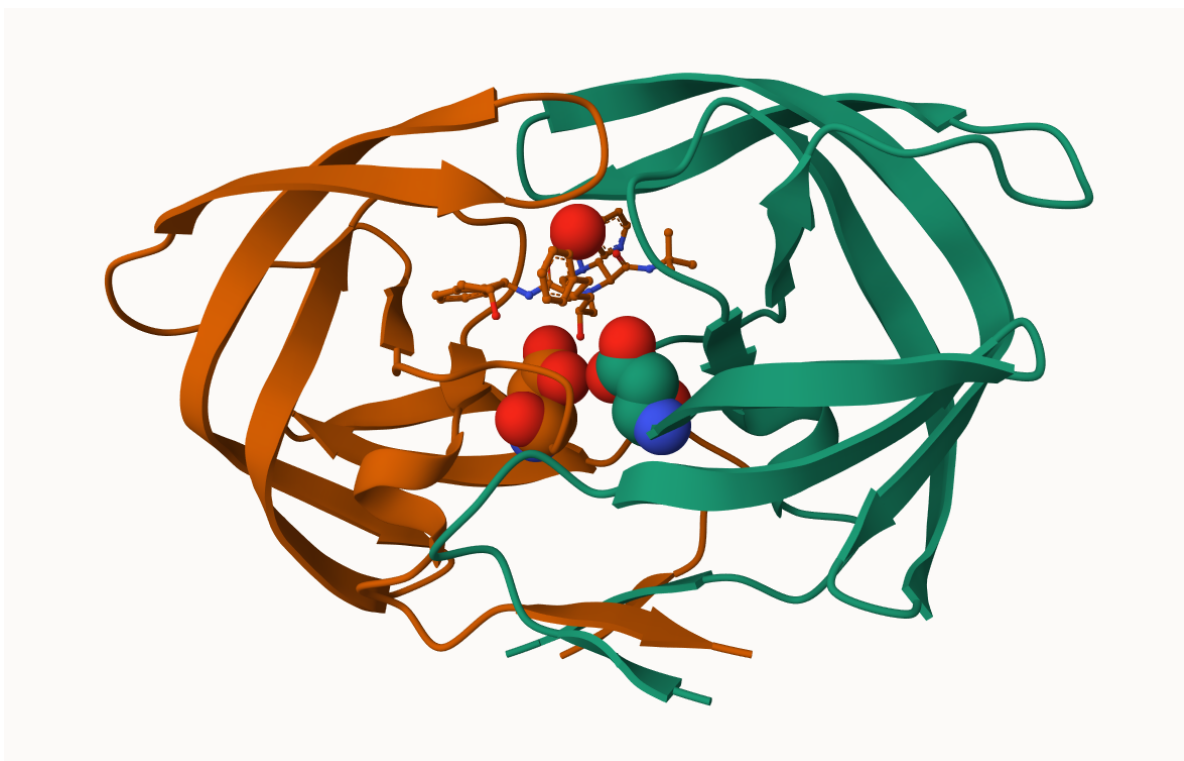


Figure 3: Spacefill display of catalytic ASP25 amino acids and key binding site water molecule

Q4: Water molecules normally have 3 atoms. Why do we see just one atom per water molecule in this structure?

Hydrogen atoms are usually not visible in X-ray structures because at typical resolutions (~ 2 Å), their electron density is too weak to detect, so only the oxygen atom of water is shown.

Q5: There is a critical “conserved” water molecule in the binding site. Can you identify this water molecule? What residue number does this water molecule have

Residue 308

Q6: Generate and save a figure clearly showing the two distinct chains of HIV-protease along with the ligand. You might also consider showing the catalytic residues ASP 25 in each chain and the critical water (we recommend “Ball & Stick” for these side-chains). Add this figure to your Quarto document.

See figures above

Getting started with the Bio3d package

Bio3D is an R package from CRAN for structural bioinformatics

```
library(bio3d)

pdb <- read.pdb("1hsg")
```

Note: Accessing on-line PDB file

```
pdb
```

```
Call: read.pdb(file = "1hsg")
```

```
Total Models#: 1
Total Atoms#: 1686, XYZs#: 5058 Chains#: 2 (values: A B)

Protein Atoms#: 1514 (residues/Calpha atoms#: 198)
Nucleic acid Atoms#: 0 (residues/phosphate atoms#: 0)

Non-protein/nucleic Atoms#: 172 (residues: 128)
Non-protein/nucleic resid values: [ HOH (127), MK1 (1) ]
```

```
Protein sequence:
PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIGGFIKVRQYD
QILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFPQITLWQRPLVTIKIGGQLKE
ALLDTGADDTVLEEMSLPGRWKPKMIGGIGGFIKVRQYDQILIEICGHKAIGTVLVGPTP
VNIIGRNLLTQIGCTLNF
```

```
+ attr: atom, xyz, seqres, helix, sheet,
      calpha, remark, call
```

Q7: How many amino acid residues are there in this pdb object?

198

Q8: Name one of the two non-protein residues?

HOH

Q9: How many protein chains are in this structure?

2 chains

```
attributes(pdb)
```

```
$names
```

```
[1] "atom" "xyz" "seqres" "helix" "sheet" "calpha" "remark" "call"
```

```
$class
```

```
[1] "pdb" "sse"
```

```
head(pdb$atom)
```

	type	eleno	elety	alt	resid	chain	resno	insert	x	y	z	o	b
1	ATOM	1	N	<NA>	PRO	A	1	<NA>	29.361	39.686	5.862	1	38.10
2	ATOM	2	CA	<NA>	PRO	A	1	<NA>	30.307	38.663	5.319	1	40.62
3	ATOM	3	C	<NA>	PRO	A	1	<NA>	29.760	38.071	4.022	1	42.64
4	ATOM	4	O	<NA>	PRO	A	1	<NA>	28.600	38.302	3.676	1	43.40
5	ATOM	5	CB	<NA>	PRO	A	1	<NA>	30.508	37.541	6.342	1	37.87
6	ATOM	6	CG	<NA>	PRO	A	1	<NA>	29.296	37.591	7.162	1	38.40

	segid	elesy	charge
1	<NA>	N	<NA>
2	<NA>	C	<NA>
3	<NA>	C	<NA>
4	<NA>	O	<NA>
5	<NA>	C	<NA>
6	<NA>	C	<NA>

There are lots of functions that can work with these `pdb` objects:

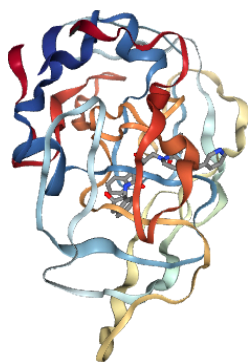
```
head(pdbseq(pdb))
```

```
  1    2    3    4    5    6  
"P" "Q" "I" "T" "L" "W"
```

We can have a quick interactive view of any of these `pdb` objects:

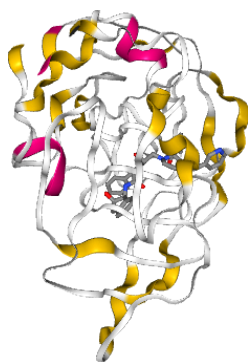
```
library(bio3dview)
```

```
view.pdb(pdb)
```



Let's try a custom view

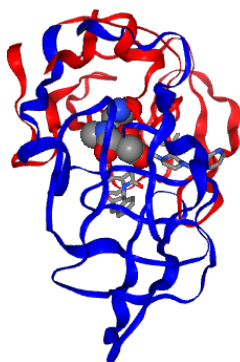
```
view.pdb(pdb,  
         colorScheme="sse",  
         backgroundColor = "black")
```



Q. Create a custom view highlighting the active site ASP (`resno=25`), the two chains (in your choice of colors) and the ligand all on a custom background.

```
library(NGLVieweR)
active.site <- atom.select(pdb, resno=25)

view.pdb(pdb,
  cols = c("red", "blue"),
  highlight = active.site,
  highlight.style = "spacefill",
  backgroundColor = "pink") |>
  setRock()
```



Predict the flexibility of a given structure

Let's do a Normal Mode Analysis (NMA) to predict the flexibility of a given `pdb` object:

```
adk <- read.pdb("6s36")
```

Note: Accessing on-line PDB file
PDB has ALT records, taking A only, `rm.alt=TRUE`

```
adk
```

```
Call: read.pdb(file = "6s36")
```

```
Total Models#: 1
Total Atoms#: 1898, XYZs#: 5694 Chains#: 1 (values: A)

Protein Atoms#: 1654 (residues/Calpha atoms#: 214)
Nucleic acid Atoms#: 0 (residues/phosphate atoms#: 0)

Non-protein/nucleic Atoms#: 244 (residues: 244)
Non-protein/nucleic resid values: [ CL (3), HOH (238), MG (2), NA (1) ]
```

Protein sequence:

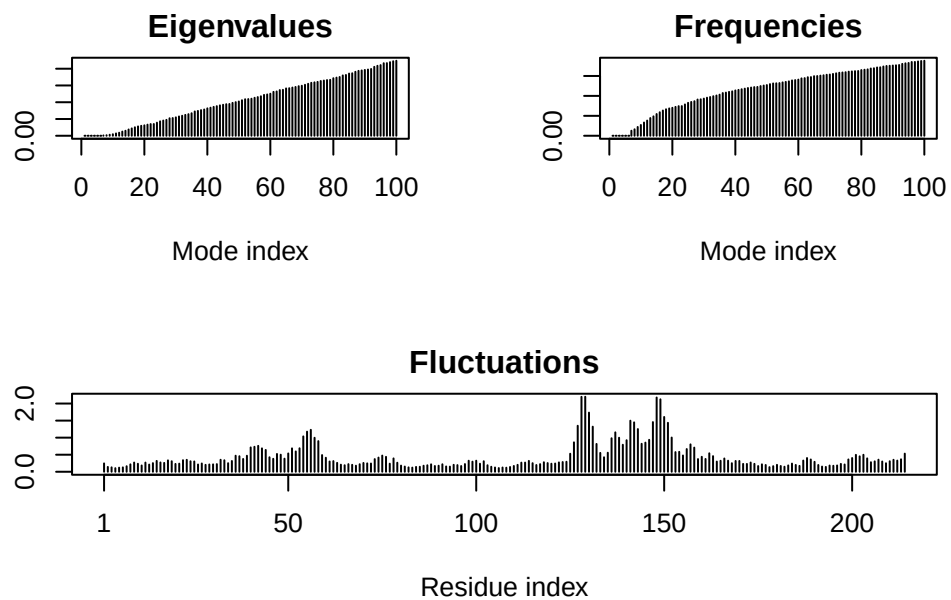
```
MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMLRAAVKSGSELGKQAKDIMDAGKLV
TDELVIALVKERIAQEDCRNGFLLDGFPR TIPQADAMKEAGINVDYVLEFDVPDELIVDKI
VGRRVHAPSGRVYHV KFNPPKVEGKDDVTGEELTTRKDDQEETVRKRLVEYHQMTAPLIG
YYSKEAEAGNTKYAKVDGTPVAEVRADLEKILG
```

```
+ attr: atom, xyz, seqres, helix, sheet,
      calpha, remark, call
```

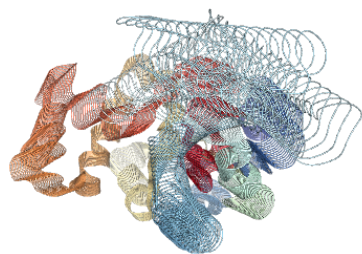
```
m <- nma(adk)
```

```
Building Hessian...      Done in 0.014 seconds.
Diagonalizing Hessian... Done in 0.294 seconds.
```

```
plot(m)
```



```
view.nma(m)
```

Write out the results for viewing in Mol-star:

```
mktrj(m, file="nma.pdb")
```

Comparative analysis of the ADK family

We will begin by first installing the packages we need for today's session.

Install packages in the R console NOT your Rmd/Quarto file

```
install.packages("bio3d") install.packages("NGLVieweR")
```

```
install.packages("remotes") remotes::install_github("bioboot/bio3dview")
```

```
install.packages("BiocManager") BiocManager::install("msa")
```

Q10. Which of the packages above is found only on BioConductor and not CRAN?

msa

Q11. Which of the above packages is not found on BioConductor or CRAN?:

bio3dview

Q12: True or False? Functions from the pak package can be used to install packages from GitHub and BitBucket?

True

Our first step is find a sequence for this family. We will use the database ID "1ake_A" here:

```
id <- "1ake_A"  
aa <- get.seq(id)
```

Warning in get.seq(id): Removing existing file: seqs.fasta

Fetching... Please wait. Done.

```
aa
```

```

      1      .      .      .      .      .      .      60
pdb|1AKE|A  MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMLRAAVKSGSELGKQAKDIMDAGKLV
      1      .      .      .      .      .      .      60

      61      .      .      .      .      .      .      120
pdb|1AKE|A  DELVIALVKERIAQEDCRNGFLLDGFPRTPQADAMKEAGINVDYVLEFDVPDELIVDRI
      61      .      .      .      .      .      .      120

      121      .      .      .      .      .      .      180
pdb|1AKE|A  VGRRVHAPSGRVYHVKNPPKVEGKDDVTGEELTTRKDDQEETVRKRLVEYHQMTPALIG
      121      .      .      .      .      .      .      180

      181      .      .      .      214
pdb|1AKE|A  YYSKEAEAGNTKYAKVDGTPVAEVRADLEKILG
      181      .      .      .      214

```

Call:

```
read.fasta(file = outfile)
```

Class:

```
fasta
```

Alignment dimensions:

```
1 sequence rows; 214 position columns (214 non-gap, 0 gap)
```

+ attr: id, ali, call

Q13. How many amino acids are in this sequence, i.e. how long is this sequence?

214

Search for related sequences in the database

```
blast <- blast.pdb(aa)
```

```
Searching ... please wait (updates every 5 seconds) RID = ZN34RPT2014
```

```
...
```

```
Reporting 96 hits
```

```
head(blast$hit.tbl)
```

```
queryid subjectids identity alignmentlength mismatches gapopens q.start
```

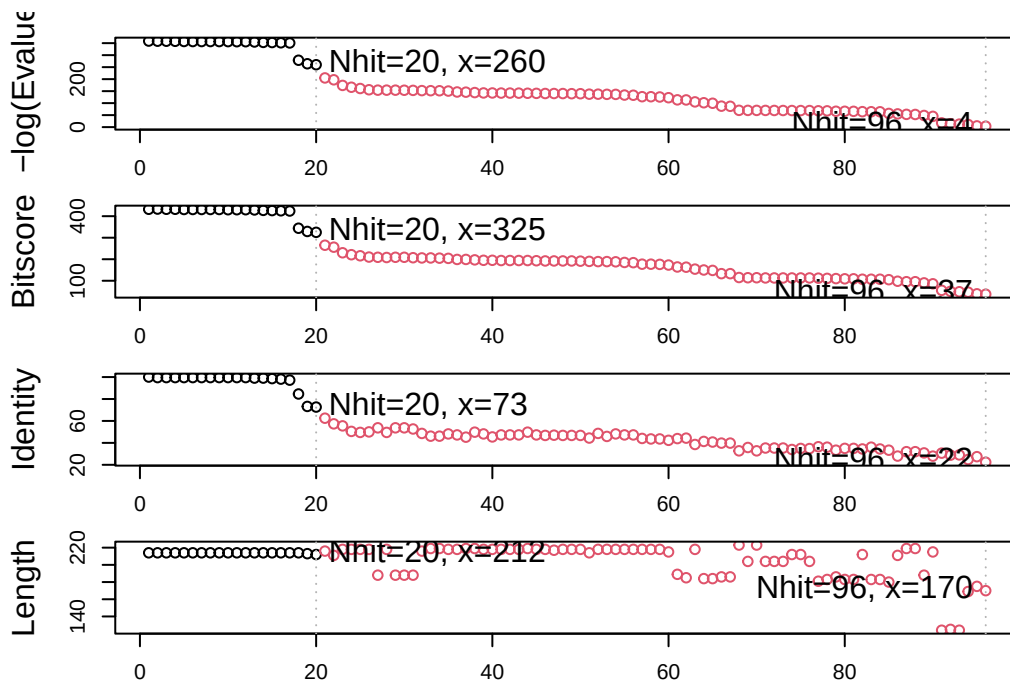
1	Query_7927671	1AKE_A	100.000	214	0	0	1
2	Query_7927671	8BQF_A	99.533	214	1	0	1
3	Query_7927671	4X8M_A	99.533	214	1	0	1
4	Query_7927671	6S36_A	99.533	214	1	0	1
5	Query_7927671	9R6U_A	99.533	214	1	0	1
6	Query_7927671	9R71_A	99.533	214	1	0	1

	q.end	s.start	s.end	evalue	bitscore	positives	mlog.evalue	pdb.id	acc
1	214	1	214	1.81e-156	432	100.00	358.6099	1AKE_A	1AKE_A
2	214	21	234	2.97e-156	433	100.00	358.1147	8BQF_A	8BQF_A
3	214	1	214	3.25e-156	432	100.00	358.0246	4X8M_A	4X8M_A
4	214	1	214	4.76e-156	432	100.00	357.6430	6S36_A	6S36_A
5	214	1	214	1.06e-155	431	99.53	356.8424	9R6U_A	9R6U_A
6	214	1	214	1.25e-155	431	99.53	356.6775	9R71_A	9R71_A

```
hits <-plot(blast)
```

```
* Possible cutoff values:    260 3
    Yielding Nhits:         20 96
```

```
* Chosen cutoff value of:    260
    Yielding Nhits:         20
```



```
hits$pdb.id
```

```
[1] "1AKE_A" "8BQF_A" "4X8M_A" "6S36_A" "9R6U_A" "9R71_A" "8Q2B_A" "8RJ9_A"  
[9] "6RZE_A" "4X8H_A" "3HPR_A" "1E4V_A" "5EJE_A" "1E4Y_A" "3X2S_A" "6HAP_A"  
[17] "6HAM_A" "8PVW_A" "4K46_A" "4NP6_A"
```

```
files <- get.pdb(hits$pdb.id, path = "pdbc", split=TRUE, gzip=TRUE)
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbc", split = TRUE, gzip = TRUE):  
pdbc/1AKE.pdb.gz exists. Skipping download
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbc", split = TRUE, gzip = TRUE):  
pdbc/8BQF.pdb.gz exists. Skipping download
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbc", split = TRUE, gzip = TRUE):  
pdbc/4X8M.pdb.gz exists. Skipping download
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbc", split = TRUE, gzip = TRUE):  
pdbc/6S36.pdb.gz exists. Skipping download
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbc", split = TRUE, gzip = TRUE):  
pdbc/9R6U.pdb.gz exists. Skipping download
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbc", split = TRUE, gzip = TRUE):  
pdbc/9R71.pdb.gz exists. Skipping download
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbc", split = TRUE, gzip = TRUE):  
pdbc/8Q2B.pdb.gz exists. Skipping download
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbc", split = TRUE, gzip = TRUE):  
pdbc/8RJ9.pdb.gz exists. Skipping download
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbc", split = TRUE, gzip = TRUE):  
pdbc/6RZE.pdb.gz exists. Skipping download
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbc", split = TRUE, gzip = TRUE):  
pdbc/4X8H.pdb.gz exists. Skipping download
```

Warning in get.pdb(hits\$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/3HPR.pdb.gz exists. Skipping download

Warning in get.pdb(hits\$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/1E4V.pdb.gz exists. Skipping download

Warning in get.pdb(hits\$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/5EJE.pdb.gz exists. Skipping download

Warning in get.pdb(hits\$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/1E4Y.pdb.gz exists. Skipping download

Warning in get.pdb(hits\$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/3X2S.pdb.gz exists. Skipping download

Warning in get.pdb(hits\$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/6HAP.pdb.gz exists. Skipping download

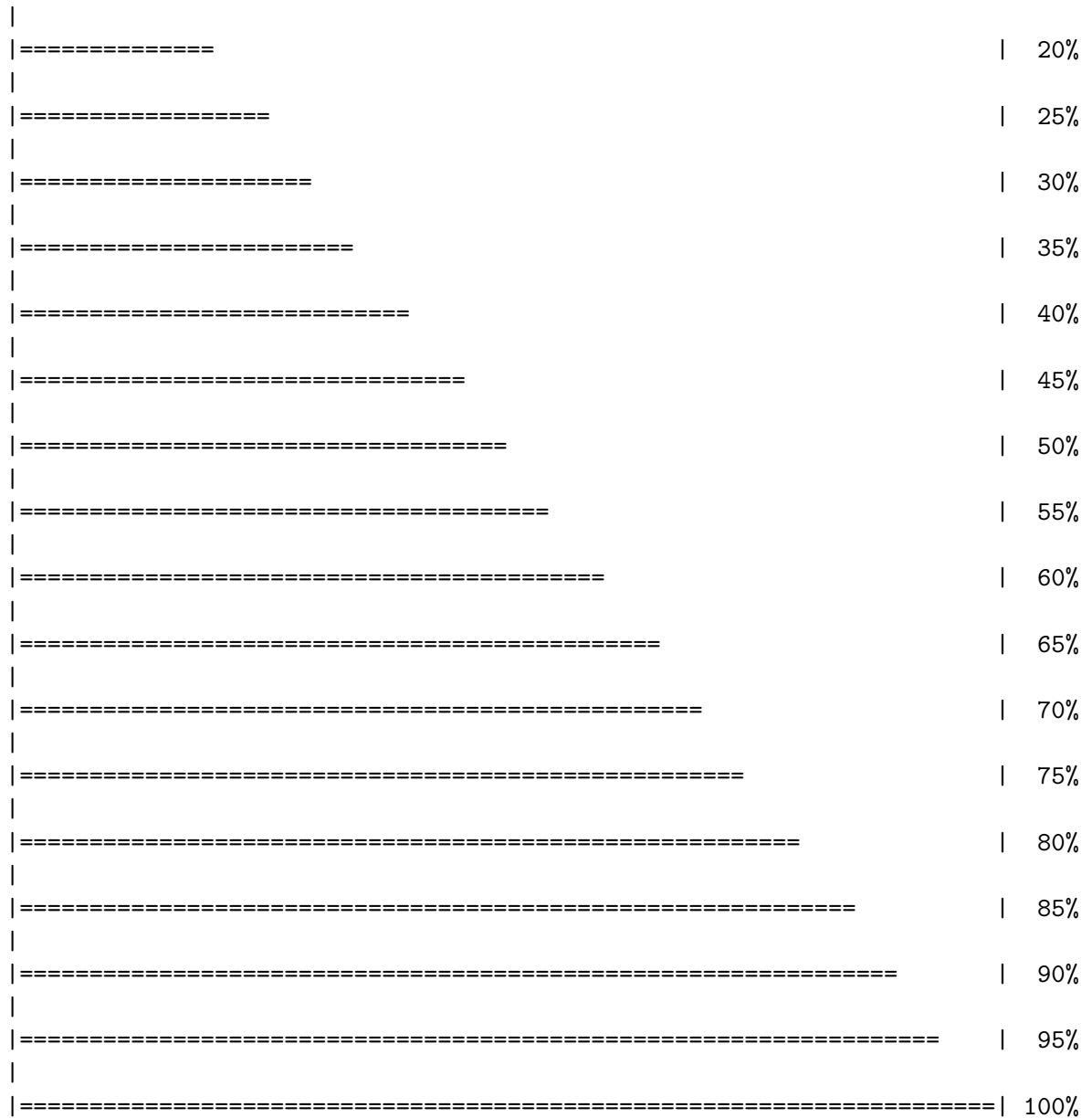
Warning in get.pdb(hits\$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/6HAM.pdb.gz exists. Skipping download

Warning in get.pdb(hits\$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/8PVW.pdb.gz exists. Skipping download

Warning in get.pdb(hits\$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/4K46.pdb.gz exists. Skipping download

Warning in get.pdb(hits\$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/4NP6.pdb.gz exists. Skipping download

	0%
====	5%
=====	10%
=====	15%



Align and superpose all these ADK structures

```
pdbbs <- pdbaln(files, fit = TRUE, exefile="msa")
```

Reading PDB files:

```
pdbbs/split_chain/1AKE_A.pdb
pdbbs/split_chain/8BQF_A.pdb
```

```

pdbs/split_chain/4X8M_A.pdb
pdbs/split_chain/6S36_A.pdb
pdbs/split_chain/9R6U_A.pdb
pdbs/split_chain/9R71_A.pdb
pdbs/split_chain/8Q2B_A.pdb
pdbs/split_chain/8RJ9_A.pdb
pdbs/split_chain/6RZE_A.pdb
pdbs/split_chain/4X8H_A.pdb
pdbs/split_chain/3HPR_A.pdb
pdbs/split_chain/1E4V_A.pdb
pdbs/split_chain/5EJE_A.pdb
pdbs/split_chain/1E4Y_A.pdb
pdbs/split_chain/3X2S_A.pdb
pdbs/split_chain/6HAP_A.pdb
pdbs/split_chain/6HAM_A.pdb
pdbs/split_chain/8PVW_A.pdb
pdbs/split_chain/4K46_A.pdb
pdbs/split_chain/4NP6_A.pdb
    PDB has ALT records, taking A only, rm.alt=TRUE
.    PDB has ALT records, taking A only, rm.alt=TRUE
..   PDB has ALT records, taking A only, rm.alt=TRUE
.    PDB has ALT records, taking A only, rm.alt=TRUE
.    PDB has ALT records, taking A only, rm.alt=TRUE
.    PDB has ALT records, taking A only, rm.alt=TRUE
.    PDB has ALT records, taking A only, rm.alt=TRUE
.    PDB has ALT records, taking A only, rm.alt=TRUE
..   PDB has ALT records, taking A only, rm.alt=TRUE
..   PDB has ALT records, taking A only, rm.alt=TRUE
....  PDB has ALT records, taking A only, rm.alt=TRUE
.    PDB has ALT records, taking A only, rm.alt=TRUE
.    PDB has ALT records, taking A only, rm.alt=TRUE
..

```

Extracting sequences

```

pdb/seq: 1    name: pdbs/split_chain/1AKE_A.pdb
    PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 2    name: pdbs/split_chain/8BQF_A.pdb
    PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 3    name: pdbs/split_chain/4X8M_A.pdb
pdb/seq: 4    name: pdbs/split_chain/6S36_A.pdb
    PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 5    name: pdbs/split_chain/9R6U_A.pdb

```


PDB has ALT records, taking A only, rm.alt=TRUE
 pdb/seq: 6 name: pdbc/split_chain/9R71_A.pdb
 PDB has ALT records, taking A only, rm.alt=TRUE
 pdb/seq: 7 name: pdbc/split_chain/8Q2B_A.pdb
 PDB has ALT records, taking A only, rm.alt=TRUE
 pdb/seq: 8 name: pdbc/split_chain/8RJ9_A.pdb
 PDB has ALT records, taking A only, rm.alt=TRUE
 pdb/seq: 9 name: pdbc/split_chain/6RZE_A.pdb
 PDB has ALT records, taking A only, rm.alt=TRUE
 pdb/seq: 10 name: pdbc/split_chain/4X8H_A.pdb
 pdb/seq: 11 name: pdbc/split_chain/3HPR_A.pdb
 PDB has ALT records, taking A only, rm.alt=TRUE
 pdb/seq: 12 name: pdbc/split_chain/1E4V_A.pdb
 pdb/seq: 13 name: pdbc/split_chain/5EJE_A.pdb
 PDB has ALT records, taking A only, rm.alt=TRUE
 pdb/seq: 14 name: pdbc/split_chain/1E4Y_A.pdb
 pdb/seq: 15 name: pdbc/split_chain/3X2S_A.pdb
 pdb/seq: 16 name: pdbc/split_chain/6HAP_A.pdb
 pdb/seq: 17 name: pdbc/split_chain/6HAM_A.pdb
 PDB has ALT records, taking A only, rm.alt=TRUE
 pdb/seq: 18 name: pdbc/split_chain/8PVW_A.pdb
 PDB has ALT records, taking A only, rm.alt=TRUE
 pdb/seq: 19 name: pdbc/split_chain/4K46_A.pdb
 PDB has ALT records, taking A only, rm.alt=TRUE
 pdb/seq: 20 name: pdbc/split_chain/4NP6_A.pdb

pdbc

	1	40
[Truncated_Name:1] 1AKE_A.pdb	--MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMRLAA	
[Truncated_Name:2] 8BQF_A.pdb	--MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMRLAA	
[Truncated_Name:3] 4X8M_A.pdb	--MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMRLAA	
[Truncated_Name:4] 6S36_A.pdb	--MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMRLAA	
[Truncated_Name:5] 9R6U_A.pdb	--MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMRLAA	
[Truncated_Name:6] 9R71_A.pdb	--MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMRLAA	
[Truncated_Name:7] 8Q2B_A.pdb	--MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMRLAA	
[Truncated_Name:8] 8RJ9_A.pdb	--MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMRLAA	
[Truncated_Name:9] 6RZE_A.pdb	--MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMRLAA	
[Truncated_Name:10] 4X8H_A.pdb	--MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMRLAA	
[Truncated_Name:11] 3HPR_A.pdb	--MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMRLAA	
[Truncated_Name:12] 1E4V_A.pdb	--MRIILLGAPVAGKGTQAQFIMEKYGIPQISTGDMRLAA	
[Truncated_Name:13] 5EJE_A.pdb	--MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMRLAA	

[Truncated_Name:9] 6RZE_A.pdb	NGFLLDGFPR TIPQADAMKEAGINVDYVLEFDVPDELIVD
[Truncated_Name:10] 4X8H_A.pdb	NGFLLDGFPR TIPQADAMKEAGINVDYVLEFDVPDELIVD
[Truncated_Name:11] 3HPR_A.pdb	NGFLLDGFPR TIPQADAMKEAGINVDYVLEFDVPDELIVD
[Truncated_Name:12] 1E4V_A.pdb	NGFLLDGFPR TIPQADAMKEAGINVDYVLEFDVPDELIVD
[Truncated_Name:13] 5EJE_A.pdb	NGFLLDGFPR TIPQADAMKEAGINVDYVLEFDVPDELIVD
[Truncated_Name:14] 1E4Y_A.pdb	NGFLLDGFPR TIPQADAMKEAGINVDYVLEFDVPDELIVD
[Truncated_Name:15] 3X2S_A.pdb	NGFLLDGFPR TIPQADAMKEAGINVDYVLEFDVPDELIVD
[Truncated_Name:16] 6HAP_A.pdb	NGFLLDGFPR TIPQADAMKEAGINVDYVLEFDVPDELIVD
[Truncated_Name:17] 6HAM_A.pdb	NGFLLDGFPR TIPQADAMKEAGINVDYVLEFDVPDELIVD
[Truncated_Name:18] 8PVW_A.pdb	NGFLLDGFPR TIPQADAMKEAGINVDYVLEFDVPDELIVD
[Truncated_Name:19] 4K46_A.pdb	KGFLLDGFPR TIPQADGLKEVGVVVDYVIEFDVADSVIVE
[Truncated_Name:20] 4NP6_A.pdb	KGFLLDGFPR TIPQADGLKEMGINVDYVIEFDVADDVIVE
	**** *****^~** *^ ****^***** * ^**^
	81 . . . 120
	121 . . . 160
[Truncated_Name:1] 1AKE_A.pdb	RIVGRRVHAPSGRVYHV KFNPPKVEGKDDVTGEELTTRKD
[Truncated_Name:2] 8BQF_A.pdb	RIVGRRVHAPSGRVYHV KFNPPKVEGKDDVTGEELTTRKD
[Truncated_Name:3] 4X8M_A.pdb	RIVGRRVHAPSGRVYHV KFNPPKVEGKDDVTGEELTTRKD
[Truncated_Name:4] 6S36_A.pdb	KIVGRRVHAPSGRVYHV KFNPPKVEGKDDVTGEELTTRKD
[Truncated_Name:5] 9R6U_A.pdb	RIVGRRVHAPSGRVYHV KFNPPKVEGKDDVTGEELTTRKD
[Truncated_Name:6] 9R71_A.pdb	RIVGRRVHAPSGRVYHV KFNPPKVEGKDDVTGEELTTRKD
[Truncated_Name:7] 8Q2B_A.pdb	RIVGRRVHAPSGRVYHV KFNPPKVEGKDDVTGEELTTRKA
[Truncated_Name:8] 8RJ9_A.pdb	RIVGRRVHAPSGRVYHV KFNPPKVEGKDDVTGEELTTRKD
[Truncated_Name:9] 6RZE_A.pdb	AIVGRRVHAPSGRVYHV KFNPPKVEGKDDVTGEELTTRKD
[Truncated_Name:10] 4X8H_A.pdb	RIVGRRVHAPSGRVYHV KFNPPKVEGKDDVTGEELTTRKD
[Truncated_Name:11] 3HPR_A.pdb	RIVGRRVHAPSGRVYHV KFNPPKVEGKDDGTGEELTTRKD
[Truncated_Name:12] 1E4V_A.pdb	RIVGRRVHAPSGRVYHV KFNPPKVEGKDDVTGEELTTRKD
[Truncated_Name:13] 5EJE_A.pdb	RIVGRRVHAPSGRVYHV KFNPPKVEGKDDVTGEELTTRKD
[Truncated_Name:14] 1E4Y_A.pdb	RIVGRRVHAPSGRVYHV KFNPPKVEGKDDVTGEELTTRKD
[Truncated_Name:15] 3X2S_A.pdb	RIVGRRVHAPSGRVYHV KFNPPKVEGKDDVTGEELTTRKD
[Truncated_Name:16] 6HAP_A.pdb	RIVGRRVHAPSGRVYHV KFNPPKVEGKDDVTGEELTTRKD
[Truncated_Name:17] 6HAM_A.pdb	RIVGRRVHAPSGRVYHV KFNPPKVEGKDDVTGEELTTRKD
[Truncated_Name:18] 8PVW_A.pdb	RILKR--GETSGRV-----D
[Truncated_Name:19] 4K46_A.pdb	RMAGRRAHLASGR TYHNVYNPPKVEGKDDVTGEDLVIRE
[Truncated_Name:20] 4NP6_A.pdb	RMAGRRAHLPSGR TYHVYNPPKVEGKDDVTGEDLVIRE
	^ * ***
	121 . . . 160
	161 . . . 200
[Truncated_Name:1] 1AKE_A.pdb	DQEETVRKRLVEYHQMTAPLIGYYSKEAEAGNTKYAKVDG
[Truncated_Name:2] 8BQF_A.pdb	DQEETVRKRLVEYHQMTAPLIGYYSKEAEAGNTKYAKVDG
[Truncated_Name:3] 4X8M_A.pdb	DQEETVRKRLVEYHQMTAPLIGYYSKEAEAGNTKYAKVDG

[Truncated_Name:4] 6S36_A.pdb	DQEETVRKRLVEYHQM TAPLIGYYSKEAEAGNTKYAKVDG
[Truncated_Name:5] 9R6U_A.pdb	DQEETVRKRLVEYHQM TAPLIGYYSKEAEAGNTKYAKVDG
[Truncated_Name:6] 9R71_A.pdb	DQEETVRKRLVEYHQM TAPLIGYYSKEAEAGNTKYAKVDG
[Truncated_Name:7] 8Q2B_A.pdb	DQEETVRKRLVEYHQM TAPLIGYYSKEAEAGNTKYAKVDG
[Truncated_Name:8] 8RJ9_A.pdb	DQEETVRKRLVEYHQM TAPLIGYYSKEAEAGNTKYAKVDG
[Truncated_Name:9] 6RZE_A.pdb	DQEETVRKRLVEYHQM TAPLIGYYSKEAEAGNTKYAKVDG
[Truncated_Name:10] 4X8H_A.pdb	DQEETVRKRLVEYHQM TAPLIGYYSKEAEAGNTKYAKVDG
[Truncated_Name:11] 3HPR_A.pdb	DQEETVRKRLVEYHQM TAPLIGYYSKEAEAGNTKYAKVDG
[Truncated_Name:12] 1E4V_A.pdb	DQEETVRKRLVEYHQM TAPLIGYYSKEAEAGNTKYAKVDG
[Truncated_Name:13] 5EJE_A.pdb	DQEECVRKRLVEYHQM TAPLIGYYSKEAEAGNTKYAKVDG
[Truncated_Name:14] 1E4Y_A.pdb	DQEETVRKRLVEYHQM TAPLIGYYSKEAEAGNTKYAKVDG
[Truncated_Name:15] 3X2S_A.pdb	DQEETVRKRLCEYHQM TAPLIGYYSKEAEAGNTKYAKVDG
[Truncated_Name:16] 6HAP_A.pdb	DQEETVRKRLVEYHQM TAPLIGYYSKEAEAGNTKYAKVDG
[Truncated_Name:17] 6HAM_A.pdb	DQEETVRKRLVEYHQM TAPLIGYYSKEAEAGNTKYAKVDG
[Truncated_Name:18] 8PVW_A.pdb	DNEETVRKRLVEYHQM TAPLIGYYSKEAEAGNTKYAKVDG
[Truncated_Name:19] 4K46_A.pdb	DKEETVLARLGVYHNQTAPLIAYYGKEAEAGNTQYLKFDG
[Truncated_Name:20] 4NP6_A.pdb	DKEETVRARLNVYHTQTAPLIEYYGKEAAAGKTQYLKFDG
	* * * * * ^ * * * * * * * * * *
161	200
	201 . 216
[Truncated_Name:1] 1AKE_A.pdb	TKPVAEVRADLEKILG
[Truncated_Name:2] 8BQF_A.pdb	TKPVAEVRADLEKIL-
[Truncated_Name:3] 4X8M_A.pdb	TKPVAEVRADLEKILG
[Truncated_Name:4] 6S36_A.pdb	TKPVAEVRADLEKILG
[Truncated_Name:5] 9R6U_A.pdb	TKPVAEVRADLEKILG
[Truncated_Name:6] 9R71_A.pdb	TKPVAEVRADLEKILG
[Truncated_Name:7] 8Q2B_A.pdb	TKPVAEVRADLEKILG
[Truncated_Name:8] 8RJ9_A.pdb	TKPVAEVRADLEKILG
[Truncated_Name:9] 6RZE_A.pdb	TKPVAEVRADLEKILG
[Truncated_Name:10] 4X8H_A.pdb	TKPVAEVRADLEKILG
[Truncated_Name:11] 3HPR_A.pdb	TKPVAEVRADLEKILG
[Truncated_Name:12] 1E4V_A.pdb	TKPVAEVRADLEKILG
[Truncated_Name:13] 5EJE_A.pdb	TKPVAEVRADLEKILG
[Truncated_Name:14] 1E4Y_A.pdb	TKPVAEVRADLEKILG
[Truncated_Name:15] 3X2S_A.pdb	TKPVAEVRADLEKILG
[Truncated_Name:16] 6HAP_A.pdb	TKPVCEVRADLEKILG
[Truncated_Name:17] 6HAM_A.pdb	TKPVCEVRADLEKILG
[Truncated_Name:18] 8PVW_A.pdb	TKPVAEVRADLEKILG
[Truncated_Name:19] 4K46_A.pdb	TKA VA EVS AELEKALA
[Truncated_Name:20] 4NP6_A.pdb	TKQVSEVSADIKALA
	** * * * ^ ^ * *
201	216

Call:

```
pdbaln(files = files, fit = TRUE, exefile = "msa")
```

Class:

```
pdbs, fasta
```

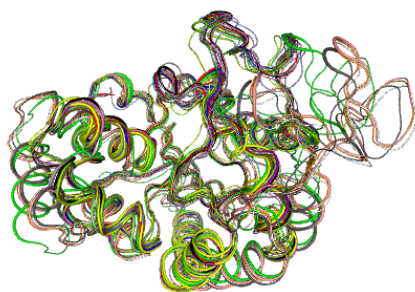
Alignment dimensions:

```
20 sequence rows; 216 position columns (182 non-gap, 34 gap)
```

```
+ attr: xyz, resno, b, chain, id, ali, resid, sse, call
```

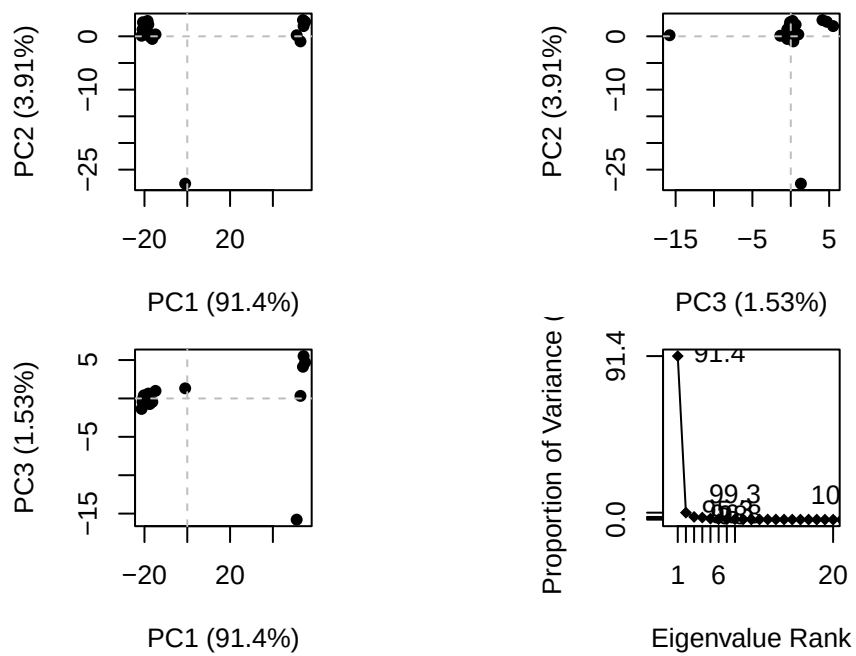
Quick interactive structural view

```
view.pdbs(pdbs)
```

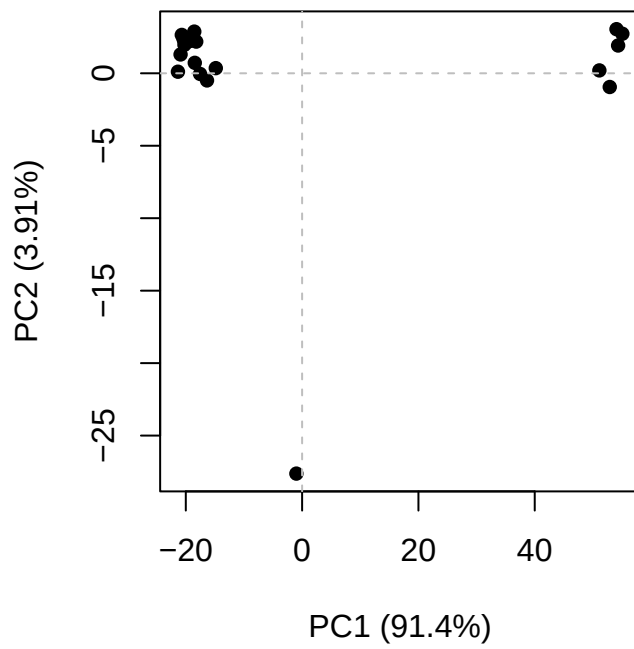


PCA of all this structural data (x, y and z atom coordinates):

```
pc <- pca(pdfs)
plot(pc)
```

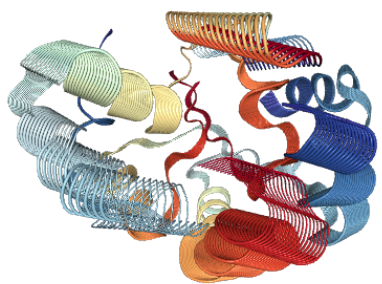


```
plot(pc, 1:2)
```



Interactive view of the PC1 captured structural differences

```
view.pca(pc)
```

```
mktrj(pc, file = "pca.pdb")
```